

El Obsoletismo del Curso INF134 - Estructuras Discretas en la PUCP: Un Análisis Crítico Comparativo

Fecha de Elaboración: 2026-03-22

Resumen Ejecutivo

Este informe presenta un análisis crítico y comparativo del curso INF134 - Estructuras Discretas, impartido en la Facultad de Ciencias e Ingeniería de la Pontificia Universidad Católica del Perú (PUCP). El objetivo central es defender la tesis de que el diseño curricular, la metodología pedagógica y el sistema de evaluación del curso se han vuelto obsoletos, contraviniendo las mejores prácticas internacionales y fallando en el desarrollo de las competencias fundamentales que la ciencia de la computación moderna demanda. Utilizando exclusivamente la información contenida en el sílabo detallado del curso, junto con análisis comparativos de estándares internacionales (ACM/IEEE), universidades de élite, el contexto educativo peruano y la literatura pedagógica contemporánea, este documento evidencia las profundas deficiencias del modelo actual [6].

La investigación revela que INF134 opera como un “mega-curso” de una amplitud temática excepcional, fusionando en una sola asignatura de 4 créditos lo que internacionalmente y en otras universidades peruanas de referencia constituyen al menos tres áreas de estudio distintas: matemáticas discretas puras, diseño lógico digital y teoría de la computación [6]. Esta estructura enciclopédica, si bien ambiciosa, conduce a una cobertura superficial de los temas y genera una presión insostenible sobre el proceso de aprendizaje.

El análisis pedagógico demuestra que la metodología del curso, centrada en la “exposición magistral” y un sistema de evaluación donde el 60% de la calificación depende de dos exámenes de alto impacto (parcial y final), fomenta un aprendizaje pasivo y promueve la memorización de procedimientos por sobre la comprensión conceptual profunda [6]. Este enfoque es diametralmente opuesto a los estándares globales promovidos por la ACM/IEEE, que abogan por un aprendizaje activo, basado en competencias y con una fuerte integración de la teoría con la práctica a través de la programación [7, 9]. El benchmarking con instituciones como MIT, Stanford y CMU expone una brecha abismal en términos de innovación pedagógica, fomento de la colaboración y sistemas de evaluación diseñados para medir el dominio real y no la capacidad de regurgitación bajo presión [1, 2, 3].

Las consecuencias de este modelo obsoleto son graves: genera en los estudiantes un alto nivel de estrés, una comprensión frágil y superficial de conceptos fundamentales, y una incapacidad para transferir el conocimiento teórico a la resolución de problemas prácticos. En última instancia, el curso no logra cumplir su rol vertebrador en la formación de ingenieros informáticos capaces de pensar de manera abstracta, lógica y creativa. Este informe concluye con una serie de recomendaciones urgentes para la modernización del curso, incluyendo una reestructuración curricular para dividir su contenido, una reforma pedagógica hacia el aprendizaje activo y una modernización del sistema de evaluación para priorizar el trabajo continuo y la aplicación práctica del conocimiento [1, 3, 13, 14].

Introducción - Planteamiento de la tesis del obsoletismo

El curso INF134 - Estructuras Discretas es catalogado dentro de la malla curricular de Ingeniería Informática de la PUCP como una materia fundamental y de carácter “vertebrador” [6]. Su propósito declarado es proporcionar las bases matemáticas y lógicas indispensables para la formación de un profesional en ciencias de la computación, sentando los cimientos para cursos avanzados en algoritmia, arquitectura de computadoras y teoría de la computación. Sin embargo, a pesar de su importancia estratégica, el presente informe defiende la tesis de que el diseño, la metodología y la filosofía subyacente del curso INF134, en su estado actual, representan un modelo educativo obsoleto que no solo falla en alcanzar sus propios objetivos de manera efectiva, sino que activamente obstaculiza el desarrollo de las competencias críticas que definen a un informático del siglo XXI.

Esta tesis se fundamenta en un análisis multifacético que contrasta la estructura y práctica del curso con tres pilares de referencia: los estándares curriculares internacionales definidos por la Asociación de Maquinaria Computacional (ACM) y el Instituto de Ingenieros Eléctricos y Electrónicos (IEEE) [7, 9]; las prácticas pedagógicas y curriculares de las universidades líderes en el mundo en ciencias de la computación [1, 2, 3, 4]; y los principios de la pedagogía moderna respaldados por décadas de investigación en ciencias cognitivas y educación [19, 20]. La evidencia demuestra que el enfoque de INF134, caracterizado por una sobrecarga de contenido enciclopédico, una dependencia de la enseñanza magistral pasiva y un sistema de evaluación que incentiva la memorización superficial, se encuentra en directa contradicción con las tendencias globales que priorizan la comprensión conceptual profunda, el aprendizaje activo, la resolución creativa de problemas y la conexión intrínseca entre la teoría matemática y su aplicación computacional práctica [6, 7].

El obsoletismo de INF134 no es una cuestión de contenido desactualizado, ya que los temas que cubre (lógica, grafos, autómatas) siguen siendo perennes. El problema es de enfoque, de método y de finalidad. El curso parece diseñado para una era en la que la acumulación de información era un objetivo en sí mismo, en lugar de una era que demanda la habilidad de aplicar principios fundamentales para resolver problemas nuevos y complejos en un campo tecnológico en constante cambio. Este informe desglosará, de manera objetiva y cruda, la evidencia que sustenta esta tesis, analizando las profundas brechas entre lo que el curso es, lo que debería ser según los estándares globales, y por qué su modelo actual es pedagógicamente problemático, concluyendo con las graves consecuencias de perpetuar este sistema y las vías necesarias para su impostergable modernización.

Contexto del Curso INF134 en la PUCP - Análisis del estado actual

Para comprender la magnitud de su anacronismo, es imperativo analizar en detalle la estructura y componentes del curso INF134 - Estructuras Discretas, según lo estipulado en su documento de referencia oficial [6]. El curso, de carácter obligatorio y con una carga de 4.0 créditos, se posiciona en los ciclos iniciales de la carrera de Ingeniería Informática (3º o 4º ciclo) [6]. Su rol es explícitamente “vertebrador”, sirviendo como prerrequisito o base conceptual para asignaturas cruciales como Arquitectura de Computadoras (1ELE01), Algoritmia y Estructura de Datos (1INF27) y Algoritmos Avanzados (1INF32) [6].

El rasgo más distintivo y problemático de INF134 es su **excepcional amplitud temática**. El sílabo detalla siete unidades de contenido que, en la práctica, fusionan tres áreas de conocimiento que en la mayoría de los currículos modernos se tratan como cursos separados y especializados [6]. Primero, abarca un curso completo de **Diseño Lógico y Arquitectura de Bajo Nivel**, incluyendo sistemas de

numeración, representación de datos en complemento a dos y punto flotante, aritmética binaria, álgebras de Boole, simplificación con mapas de Karnaugh y el diseño de circuitos lógicos [6]. Segundo, contiene los elementos de un curso tradicional de **Matemáticas Discretas**, como lógica proposicional y de predicados, reglas de inferencia, técnicas de demostración como la inducción matemática, teoría de relaciones (de equivalencia y de orden), teoría de grafos y la resolución de relaciones de recurrencia [6]. Tercero, introduce una porción significativa de un curso de **Teoría de la Computación y Lenguajes Formales**, cubriendo máquinas de estado finito, minimización de autómatas, expresiones regulares, gramáticas formales y el algoritmo de Thompson para la conversión de expresiones regulares a autómatas [6].

Esta consolidación de tres disciplinas fundamentales en una sola asignatura de 17 semanas y 4 créditos es una decisión curricular que, si bien puede parecer eficiente en el papel, resulta pedagógicamente insostenible. La densidad del contenido es abrumadora, forzando un ritmo de avance que inevitablemente sacrifica la profundidad por la amplitud. El cronograma semanal es un testimonio de esta carrera contra el tiempo: temas complejos como Lógica de Predicados e Inducción Matemática se cubren en una sola semana, al igual que la totalidad de Retículos y una parte de Álgebras de Boole [6].

La metodología de enseñanza declarada se basa en una estructura tradicional: 3 horas semanales de “exposición magistral” para la teoría y 2 horas de “resolución de problemas” para la práctica [6]. Aunque se menciona el “aprendizaje basado en problemas”, la estructura general y el sistema de evaluación sugieren un modelo predominantemente pasivo. El sistema de evaluación es el pilar que refuerza el enfoque obsoleto. La nota final se compone de un 30% del promedio de cuatro prácticas calificadas, un 30% de un examen parcial y un 30% de un examen final [6]. Esto significa que un **60% de la calificación total del estudiante depende del rendimiento en dos eventos de evaluación de alto impacto y duración limitada** [6]. Esta estructura crea un poderoso incentivo para el aprendizaje estratégico y superficial, donde el objetivo principal del estudiante no es la comprensión profunda y duradera, sino la memorización de procedimientos y tipos de problemas específicos para sobrevivir a estas evaluaciones cruciales. El curso, por tanto, se convierte en un filtro basado en la capacidad de memorización y gestión del estrés, en lugar de un espacio para el desarrollo del razonamiento abstracto y la resolución creativa de problemas.

Estándares Internacionales ACM/IEEE - Lo que debería ser

El contraste entre el modelo de INF134 y los estándares curriculares internacionales establecidos por la ACM y la IEEE Computer Society es profundo y revelador. Estas directrices, que representan el consenso global de la academia y la industria sobre la formación en computación, han evolucionado significativamente, alejándose de los modelos basados en la mera acumulación de conocimiento hacia enfoques centrados en el desarrollo de competencias.

Las directrices de Computer Science Curricula 2013 (CS2013) ya posicionaban a las Estructuras Discretas (DS) como una de las 18 Áreas de Conocimiento (KA) fundamentales, reconociéndola como el pilar matemático esencial para casi todas las demás áreas de la computación [9]. Sin embargo, la filosofía de CS2013 ya era flexible, presentando estas áreas no como cursos rígidos, sino como unidades de conocimiento que las instituciones podían organizar de diversas maneras, fomentando la integración a lo largo del currículo [9].

La evolución más significativa llegó con el informe Computing Curricula 2020 (CC2020), titulado “Paradigmas para la Educación Global en Computación” [7]. Este documento marca un cambio filosófico fundamental, transitando de un modelo basado en el conocimiento a uno **basado en com-**

petencias [7, 8]. El objetivo ya no es simplemente que los estudiantes “sepan” un conjunto de hechos, sino que sean “capaces de hacer” cosas con ese conocimiento. CC2020 enfatiza el desarrollo de competencias transversales como el **pensamiento sistemático**, la **creatividad**, la **colaboración** y, crucialmente, el **aprendizaje mediante la práctica (learning by doing)** [7].

Desde esta perspectiva moderna, un curso de Estructuras Discretas ya no puede ser una simple exposición de definiciones y teoremas. Debe ser un taller activo para el desarrollo del pensamiento lógico y la resolución de problemas. El contenido central (lógica, conjuntos, funciones, relaciones, técnicas de demostración, combinatoria y teoría de grafos) sigue siendo el mismo, pero su propósito pedagógico se transforma. La lógica no se enseña solo para ser memorizada, sino para ser utilizada como herramienta para construir y validar argumentos, para verificar la corrección de software o para diseñar sistemas inteligentes. La teoría de grafos no es un conjunto de definiciones abstractas, sino un lenguaje para modelar y resolver problemas del mundo real, desde redes sociales hasta logística y bioinformática.

La crítica más severa que los estándares modernos plantean al enfoque tradicional es su dependencia de la **memorización de memoria (rote memorization)** [10, 11]. En la era digital, donde la información es instantáneamente accesible, la habilidad de recordar fórmulas bajo presión es una competencia de bajo valor. Lo que se valora es la capacidad de razonar, de aplicar principios fundamentales a problemas nuevos y de utilizar herramientas computacionales para explorar y validar hipótesis. El enfoque de CC2020 sugiere que la evaluación debe reflejar estas competencias, moviéndose de exámenes de memorización a proyectos, problemas abiertos y evaluaciones que requieran la aplicación práctica del conocimiento [7]. La integración de la programación no es un complemento, sino una parte esencial de la pedagogía, ya que permite a los estudiantes experimentar con conceptos abstractos, visualizar estructuras de datos y ver la conexión directa entre la matemática formal y el código que escriben [22, 23]. En resumen, los estándares internacionales dictan que un curso de Estructuras Discretas moderno debe ser activo, aplicado, centrado en competencias y profundamente integrado con la práctica de la computación, un modelo que contrasta directamente con la estructura pasiva y teoricista de INF134.

Benchmarking Internacional - Cómo lo hacen las mejores universidades

El análisis comparativo del curso INF134 con sus equivalentes en universidades de élite mundial como el Massachusetts Institute of Technology (MIT), Stanford, Carnegie Mellon (CMU) y UC Berkeley, no solo confirma la tesis del obsoletismo, sino que también ilumina el camino hacia la modernización [1, 2, 3, 4]. Estas instituciones, que marcan la pauta en la educación en ciencias de la computación, han abandonado hace tiempo el modelo enciclopédico y pasivo que aún caracteriza al curso de la PUCP.

Una primera diferencia fundamental es la **especialización y profundidad sobre la amplitud desmedida**. Ninguna de estas universidades intenta comprimir matemáticas discretas, diseño lógico y teoría de autómatas en un solo curso [1, 2, 3, 4]. El curso 6.1200J del MIT, “Mathematics for Computer Science”, se enfoca en los fundamentos matemáticos y sus aplicaciones algorítmicas y probabilísticas [1]. El CS103 de Stanford, “Mathematical Foundations of Computing”, se adentra profundamente en la teoría de la computabilidad y la complejidad, culminando en el problema P vs. NP [2]. El CS70 de Berkeley divide su temario entre matemáticas discretas aplicadas (incluyendo criptografía RSA) y una introducción rigurosa a la teoría de la probabilidad [4]. Esta especialización permite una exploración mucho más profunda y significativa de cada tema. Frente a esto, el enfoque de “gran sondeo” de INF134, si bien expone a los estudiantes a muchas áreas, lo hace a un nivel superficial que impide el dominio real de cualquiera de ellas [6].

La segunda brecha, y quizás la más importante, reside en la **metodología pedagógica y la estructura de evaluación**. Estas universidades han abrazado plenamente el aprendizaje activo. El MIT, por ejemplo, complementa sus clases magistrales con “recitaciones” (prácticas guiadas en grupos pequeños) donde la participación activa del estudiante cuenta para la nota final [1]. La evaluación se aleja drásticamente del modelo de exámenes de alto impacto. En el MIT, las tareas de problemas (“Problem Sets”) constituyen un 35% de la nota, incentivando el trabajo constante y la práctica profunda [1]. Además, implementan políticas pedagógicamente avanzadas como la “regla de ajuste de peso”, que reduce el impacto del examen con la nota más baja del estudiante, reconociendo que el aprendizaje no es lineal y mitigando el castigo por un mal día [1].

CMU, en su curso 21-127 “Concepts of Mathematics”, se enfoca explícitamente en enseñar a “pensar como un matemático”, poniendo el énfasis en el arte de la demostración [3]. Fomenta la colaboración en las tareas y utiliza un sistema de examen innovador donde el 50% de la puntuación proviene de “preguntas preparadas” que los estudiantes reciben con una semana de antelación, promoviendo el estudio colaborativo y el dominio del material sin la presión del tiempo [3]. Stanford, por su parte, pone un énfasis extraordinario en el rigor de la comunicación matemática, recomendando el uso de herramientas como LaTeX y proporcionando guías detalladas sobre cómo escribir pruebas formales, una habilidad crítica para la investigación y el desarrollo avanzado [2].

En comparación, el sistema de INF134, con sus 3 horas de teoría pasiva y su 60% de la nota concentrada en dos exámenes, parece pertenecer a una era educativa pasada [6]. Mientras las universidades de élite diseñan ecosistemas de aprendizaje que fomentan la colaboración, la práctica continua, la resiliencia ante el fracaso y el desarrollo de habilidades de comunicación técnica, INF134 perpetúa un sistema que promueve la competencia individualista, el estudio de último minuto (“cramming”) y la memorización de procedimientos. La práctica en INF134, aunque presente, se orienta hacia el diseño de hardware y compiladores, una conexión válida pero mucho más estrecha y menos general que las aplicaciones algorítmicas, probabilísticas y teóricas exploradas en los cursos de referencia internacional [6].

Comparación con el Contexto Peruano - Situación local

El análisis del curso INF134 en el contexto de otras universidades peruanas de prestigio revela que, si bien algunos de los problemas pedagógicos tradicionales no son exclusivos de la PUCP, la estructura curricular del curso es una anomalía significativa incluso a nivel nacional. La comparación con instituciones como la UPC, UTEC, UNI, UNMSM y UNFV demuestra que existen modelos alternativos dentro del propio país que buscan abordar de diferentes maneras los desafíos de enseñar esta materia fundamental [12, 13, 14, 15, 16, 17].

La conclusión más contundente es que la **estructura de “mega-curso” de INF134 es única en su desmesurada amplitud**. Ninguna otra universidad analizada intenta una fusión tan ambiciosa de temas. La Universidad de Ingeniería y Tecnología (UTEC), por ejemplo, adopta un enfoque diametralmente opuesto: dedica dos cursos semestrales, “Matemáticas Discretas I” y “Matemáticas Discretas II”, sumando un total de 8 créditos al tema [13, 14]. Esta división permite una profundización y especialización que es estructuralmente imposible en el modelo de la PUCP. La Universidad Peruana de Ciencias Aplicadas (UPC) presenta un modelo híbrido en su curso “Matemática Discreta” de 6 créditos, que combina los fundamentos de la disciplina con Álgebra Lineal, pero separa claramente la teoría de la computación y la arquitectura de computadoras en otras asignaturas [12]. La Universidad Nacional Federico Villarreal (UNFV) también ofrece un curso de 4 créditos, pero lo combina con fundamentos de programación y algoritmos, orientando su aplicación directamente hacia el software [17].

Esta divergencia estructural es clave para entender la percepción de “desconexión teoría-práctica”. Mientras que INF134 intenta forjar las conexiones internamente (de la lógica a los circuitos, de las relaciones a los autómatas), su enfoque práctico se concentra en el hardware y los compiladores [6]. Otras universidades proponen vínculos diferentes y, en algunos casos, más alineados con las expectativas de un estudiante de software. La UPC y la UNFV, por ejemplo, vinculan explícitamente el curso a la programación de aplicaciones a través de proyectos y la enseñanza de pseudocódigo [12, 17]. La UNMSM, por su parte, enfoca su curso en la aplicación práctica mediante herramientas computacionales como Matlab [16]. Esto demuestra que el problema no es la ausencia de un vínculo práctico en la PUCP, sino la naturaleza específica y quizás demasiado estrecha de ese vínculo, que puede no resonar con la totalidad del alumnado.

En cuanto a la metodología de evaluación y el fomento de la memorización, el panorama peruano es mixto. El modelo de la PUCP, con un 60% de la nota en dos exámenes, es tradicional y ciertamente no es único; la UTEC presenta una ponderación similar [6, 14]. Sin embargo, otras instituciones ya están explorando alternativas. La UPC, por ejemplo, distribuye la evaluación de manera más equitativa, asignando un 45% a trabajos y proyectos finales, reduciendo así la presión sobre los exámenes puntuales [12]. La UNFV también asigna un 40% a “Trabajos Académicos”, valorando más el proceso continuo [17]. La existencia de estos modelos dentro del propio Perú invalida cualquier argumento de que el sistema de evaluación de INF134 es el único posible o el estándar local. Demuestra que es una elección pedagógica, y una que, a la luz de la evidencia, resulta ser conservadora y subóptima.

Fundamentación Pedagógica - Por qué el enfoque actual es problemático

El enfoque pedagógico del curso INF134 es problemático no por una cuestión de opinión, sino porque contraviene principios fundamentales del aprendizaje respaldados por una vasta investigación en ciencias cognitivas y educación. La estructura actual del curso se asienta sobre pilares pedagógicos endebles: el aprendizaje pasivo, la priorización de la memorización sobre la comprensión conceptual y una desconexión funcional entre la teoría abstracta y la práctica relevante.

Primero, el modelo basado en la “exposición magistral” promueve un **aprendizaje pasivo**, donde el estudiante es un receptor de información en lugar de un constructor activo de conocimiento. La evidencia científica es abrumadora en este punto. Un metaanálisis de 225 estudios en disciplinas STEM (ciencia, tecnología, ingeniería y matemáticas) demostró que los estudiantes en entornos de aprendizaje activo no solo obtienen mejores calificaciones, sino que sus tasas de fracaso se reducen drásticamente [19]. Los estudiantes en clases magistrales tradicionales tienen 1.5 veces más probabilidades de reprobado [19]. El aprendizaje activo, que incluye discusiones, resolución colaborativa de problemas y proyectos, se alinea con la teoría constructivista y los “cuatro pilares del aprendizaje” de Stanislas Dehaene, donde el “compromiso activo” es esencial para que el cerebro genere hipótesis, las ponga a prueba y consolide el conocimiento de manera duradera [19]. La estructura de INF134, al relegar la interacción a sesiones de práctica que a menudo se convierten en meras repeticiones de ejercicios tipo, desaprovecha el potencial del aprendizaje activo [6].

Segundo, el sistema de evaluación, con su alta dependencia de exámenes de tiempo limitado, crea un ecosistema que favorece la **memorización sobre la comprensión conceptual** [10, 11, 21]. El aprendizaje de memoria (rote learning) conduce a un conocimiento frágil y superficial [21]. Los estudiantes pueden aprender a ejecutar procedimientos mecánicamente para pasar un examen, pero sin entender el “porqué” detrás de los métodos, son incapaces de aplicar ese conocimiento a problemas nuevos o ligeramente diferentes. Se convierten en lo que la literatura crítica denomina “zombis matemáticos” [11]. La comprensión conceptual, en cambio, implica un entendimiento profundo de los principios, sus interrelaciones y su lógica [20]. Es esta comprensión la que permite la transferencia de

conocimiento, una habilidad indispensable en un campo como la computación. Para fomentar esta comprensión, la pedagogía moderna aboga por tareas abiertas, enfoques basados en la indagación, el uso de representaciones visuales y, sobre todo, el discurso matemático y la colaboración, estrategias que el modelo actual de INF134 no promueve sistemáticamente.

Tercero, existe una **desconexión crítica entre la teoría y la práctica relevante**. Si bien el curso conecta la teoría con el diseño de hardware y los autómatas, esta aplicación es demasiado específica y no representa la totalidad de la práctica computacional [6]. La matemática discreta es el lenguaje del análisis de algoritmos, las bases de datos, la inteligencia artificial y la criptografía. Un enfoque pedagógico eficaz debe hacer estas conexiones explícitas y tangibles. La integración de la programación, utilizando lenguajes como Python, no debería ser un complemento opcional, sino el vehículo principal para que los estudiantes implementen, experimenten y “jueguen” con los conceptos abstractos [22]. Al programar un algoritmo de grafos o implementar una máquina de estados, la teoría cobra vida y su relevancia se vuelve innegable. La falta de una integración profunda y sistemática de la programación como herramienta de aprendizaje en INF134 es quizás su falla pedagógica más grave, dejando a los estudiantes con un conjunto de herramientas teóricas que no saben cómo utilizar en su principal medio de trabajo: el código.

Análisis Crítico Integrado - Evidencia del obsoletismo

La convergencia de los análisis anteriores —la estructura interna del curso, la comparación con estándares internacionales y locales, y la fundamentación pedagógica— permite construir un caso irrefutable sobre el obsoletismo del curso INF134. La evidencia se puede consolidar en tres ejes críticos que demuestran cómo el diseño actual del curso falla sistemáticamente en su misión educativa: la promoción de la memorización sobre la comprensión, la profunda desconexión entre teoría y práctica, y la brecha insalvable entre los ejercicios de práctica y las evaluaciones de alto impacto.

El primer y más dañino síntoma del obsoletismo es el **fomento sistémico de la memorización en detrimento de la comprensión conceptual**. Esta no es una consecuencia accidental, sino el resultado directo y predecible de combinar una carga de contenido enciclopédica con un sistema de evaluación punitivo. Con siete unidades masivas que abarcan tres disciplinas, los estudiantes se ven forzados a un ritmo de aprendizaje superficial [6]. La presión se magnifica por el hecho de que el 60% de su éxito o fracaso en el curso se decide en aproximadamente cinco horas de exámenes (parcial y final) [6]. En este entorno de alta presión y escasez de tiempo, la estrategia más racional para el estudiante no es buscar una comprensión profunda y conectada, sino identificar patrones de problemas, memorizar algoritmos de solución y practicar su ejecución mecánica. El curso se transforma en un ejercicio de reconocimiento de patrones y regurgitación de procedimientos, una antítesis del pensamiento matemático y computacional. Este modelo contrasta brutalmente con las prácticas de evaluación continua del MIT, los proyectos de la UPC o los exámenes colaborativos de CMU, todos diseñados para medir el dominio real y la capacidad de razonamiento [1, 3, 12].

El segundo eje es la **profunda desconexión entre la teoría abstracta y su aplicación práctica y relevante**. Si bien el sílabo de INF134 establece conexiones con el diseño de hardware y la teoría de compiladores, este vínculo es insuficiente y anacrónico como única aplicación práctica para un estudiante de ciencias de la computación del siglo XXI [6]. La matemática discreta es la base del análisis de la eficiencia de los algoritmos que escribirán, de las estructuras de datos que diseñarán y de los modelos de inteligencia artificial que implementarán. El curso falla al no integrar la programación como una herramienta pedagógica central [22]. Los estudiantes deberían estar escribiendo código para implementar relaciones, para visualizar grafos, para simular autómatas y para verificar empíricamente las complejidades que se analizan teóricamente. La ausencia de esta integración, una práctica estándar en los currículos modernos, deja la teoría flotando en un vacío abstracto. Los estudiantes

aprenden un lenguaje formal (las matemáticas) pero nunca se les exige que lo “hablen” en su lengua materna (el código), creando una disociación cognitiva que impide la transferencia efectiva de habilidades.

Finalmente, la estructura del curso genera una **brecha insalvable entre los ejercicios de práctica y las evaluaciones**. Las sesiones de práctica, en lugar de ser laboratorios de exploración y descubrimiento, se convierten en sesiones de entrenamiento para los exámenes. Los estudiantes aprenden a resolver un catálogo de problemas tipo, pero la vasta cantidad de temas y la naturaleza impredecible de un examen de alto impacto significan que siempre habrá una brecha entre lo que se puede practicar y lo que se puede evaluar. Esto genera una enorme ansiedad y una sensación de injusticia, donde el éxito parece depender más de la suerte (que en el examen venga un problema “conocido”) que de una comprensión sólida. Este ciclo de “enseñar para el examen” y “aprender para el examen” es un sello distintivo de un sistema educativo fallido. En lugar de desarrollar la confianza y la capacidad para enfrentar problemas desconocidos con principios fundamentales, el curso enseña a temer lo desconocido y a depender de recetas pre-digeridas, una preparación desastrosa para una carrera profesional definida por la resolución de problemas novedosos.

Consecuencias del Obsoletismo

La persistencia de un modelo educativo obsoleto como el del curso INF134 no es una deficiencia académica abstracta; tiene consecuencias tangibles y perjudiciales tanto para los estudiantes que lo cursan como para el perfil del profesional que la PUCP entrega a la sociedad. Estas consecuencias se manifiestan a nivel individual, profesional y reputacional, socavando la misión de la universidad de formar líderes y profesionales de excelencia.

Para el estudiante, las consecuencias inmediatas son un **alto nivel de estrés, ansiedad y una relación negativa con una materia fundamental**. Al enfrentarse a una cantidad de contenido inmanejable y a un sistema de evaluación que castiga en lugar de guiar, muchos estudiantes desarrollan una aversión hacia las matemáticas y la teoría, viéndolas como un obstáculo arbitrario en lugar de una herramienta poderosa. Esto puede llevar a una pérdida de confianza en sus propias habilidades, generando la falsa creencia de que “no son buenos para las matemáticas” cuando, en realidad, el sistema pedagógico es el que está fallando [10, 11]. El aprendizaje que se produce es, en el mejor de los casos, superficial y de corta duración. Los conceptos memorizados para un examen se olvidan rápidamente, dejando un vacío en la base de conocimientos que debería sustentar cursos posteriores.

A nivel profesional, las consecuencias son aún más graves. Los egresados que han pasado por este sistema pueden carecer de la **capacidad de aplicar principios matemáticos fundamentales para resolver problemas del mundo real**. Pueden ser capaces de escribir código, pero sin una comprensión profunda de la lógica, la combinatoria o la teoría de grafos, tendrán dificultades para analizar la eficiencia de sus propios algoritmos, para diseñar estructuras de datos complejas o para razonar sobre la corrección y seguridad de sus sistemas. El curso, en su estado actual, no logra cultivar el pensamiento abstracto y el rigor lógico que son indispensables para la innovación y para abordar los desafíos más complejos de la informática. En lugar de formar ingenieros capaces de diseñar soluciones desde los primeros principios, se corre el riesgo de formar técnicos que solo pueden aplicar soluciones preexistentes. Esta deficiencia representa una desventaja competitiva significativa en un mercado laboral global que valora cada vez más la capacidad de resolución de problemas creativos y el pensamiento crítico.

Finalmente, a nivel institucional, mantener un curso fundamental con un diseño tan evidentemente desactualizado tiene un **impacto negativo en la reputación y la calidad del programa de Ingeniería Informática de la PUCP**. En una era de transparencia y benchmarking global, los cur-

rículos y las metodologías de enseñanza son objeto de escrutinio. Mientras otras universidades, tanto a nivel internacional como nacional, avanzan hacia modelos pedagógicos más activos, aplicados y centrados en el estudiante, la PUCP corre el riesgo de quedarse rezagada. La experiencia negativa de los estudiantes se difunde, afectando la percepción del programa y su capacidad para atraer y retener talento. A largo plazo, la calidad de los egresados es el principal indicador del éxito de un programa académico. Si el curso “vertebrador” de la carrera está roto, la columna vertebral de la formación del ingeniero informático de la PUCP está comprometida.

Recomendaciones para Modernización

La crítica al curso INF134 no tendría valor si no estuviera acompañada de un conjunto de recomendaciones concretas, viables y basadas en la evidencia para su impostergable modernización. La transformación del curso no requiere una reinención radical desde cero, sino la adopción de mejores prácticas ya probadas y validadas por la comunidad académica global. Las siguientes recomendaciones, extraídas directamente de los análisis comparativos y la fundamentación pedagógica, ofrecen una hoja de ruta clara para alinear INF134 con los estándares del siglo XXI.

1. **Reestructuración Curricular Radical: Dividir para Profundizar.** La recomendación más urgente es dismantlar la estructura de “mega-curso”. El contenido actual de INF134 debe ser dividido en al menos dos, y preferiblemente tres, cursos distintos y más enfocados, siguiendo modelos como el de la UTEC [13, 14]. Una posible división sería:
 - **Curso 1: Fundamentos de Matemáticas Discretas (4 créditos):** Centrado en lógica, teoría de conjuntos, relaciones, funciones, técnicas de demostración, combinatoria y teoría de grafos.
 - **Curso 2: Diseño Lógico y Arquitectura de Computadoras I (4 créditos):** Cubriendo sistemas de numeración, representación de datos, álgebras de Boole, circuitos lógicos y fundamentos de la organización del computador.
 - **Curso 3: Teoría de la Computación (3-4 créditos):** Dedicado a lenguajes formales, autómatas, gramáticas y los límites de la computación.
 Esta división permitiría un ritmo de enseñanza razonable, una exploración profunda de cada tema y la posibilidad de implementar proyectos y evaluaciones más significativas en cada área.
2. **Reforma Pedagógica hacia el Aprendizaje Activo y Colaborativo.** Se debe abandonar el modelo de “exposición magistral” como método principal. Las horas de clase deben transformarse en talleres interactivos. Inspirándose en el modelo de “recitaciones” del MIT, las sesiones prácticas deben ser espacios de resolución de problemas en grupos pequeños, guiados por asistentes de docencia [1]. Se debe adoptar una política de colaboración explícita y matizada, como la de CMU, que aliente la discusión y resolución de problemas en grupo para las tareas, pero exija una redacción individual para asegurar la comprensión personal [3].
3. **Modernización Integral del Sistema de Evaluación.** El peso desproporcionado de los exámenes parcial y final debe ser drásticamente reducido. La evaluación debe ser continua, diversificada y orientada a la aplicación. Se propone un nuevo esquema donde los exámenes no superen el 40% de la nota final. El 60% restante debería provenir de:
 - **Tareas de problemas semanales (Problem Sets):** Con un peso significativo (ej. 30%), para incentivar el trabajo constante [1].

- **Proyectos de Programación:** Al menos uno o dos proyectos por curso donde los estudiantes deban implementar conceptos teóricos (ej. un solucionador de sudokus usando lógica, un algoritmo de camino más corto en un grafo, un simulador de autómatas).
- **Participación y Evaluaciones Cortas (Quizzes):** Para asegurar la preparación y el compromiso continuo.
Además, se podría experimentar con formatos de examen innovadores como las “preguntas preparadas” de CMU [3].

4. Integración Profunda y Sistemática de la Programación. La programación no debe ser una ocurrencia tardía, sino el lenguaje a través del cual se explora la matemática discreta. Desde el inicio, los estudiantes deben utilizar un lenguaje como Python para escribir scripts que verifiquen leyes lógicas, manipulen conjuntos y relaciones, implementen algoritmos de grafos y construyan máquinas de estado [22]. Esto no solo hace la teoría tangible y relevante, sino que también desarrolla el pensamiento computacional y las habilidades de programación de manera sinérgica. Herramientas como Jupyter Notebooks son ideales para combinar código, texto explicativo y visualizaciones matemáticas en un solo entorno de aprendizaje.

Conclusiones

El análisis exhaustivo y comparativo presentado en este informe conduce a una conclusión ineludible: el curso INF134 - Estructuras Discretas, en su concepción y ejecución actual en la Pontificia Universidad Católica del Perú, es una reliquia pedagógica. Su estructura curricular, que fusiona de manera forzada tres disciplinas fundamentales en una sola asignatura, crea una sobrecarga de contenido que impide la comprensión profunda [6]. Su metodología, anclada en el aprendizaje pasivo de la clase magistral, y su sistema de evaluación, que prioriza la memorización para exámenes de alto impacto, contravienen directamente las prácticas educativas modernas y los estándares internacionales establecidos por la ACM/IEEE [7, 9].

La evidencia es clara: mientras las principales instituciones educativas del mundo han evolucionado hacia modelos de aprendizaje activo, colaborativo, basado en competencias y con una fuerte integración de la teoría con la práctica computacional, INF134 permanece estancado en un paradigma obsoleto [1, 3, 7]. Este anacronismo no es inocuo; genera consecuencias negativas directas en la formación de los estudiantes, limitando su capacidad para el pensamiento abstracto, la resolución creativa de problemas y la aplicación efectiva del conocimiento. Falla en su misión de ser un curso “vertebrador” y, en cambio, se convierte en un filtro basado en la resistencia al estrés y la capacidad de memorización a corto plazo.

La comparación con otras universidades, tanto a nivel internacional como dentro del propio Perú, demuestra que existen alternativas viables y más efectivas [1, 12, 13]. La modernización no es una utopía, sino una necesidad urgente y alcanzable. La reestructuración del contenido en cursos más enfocados, la adopción de metodologías de aprendizaje activo, la reforma del sistema de evaluación para valorar el proceso y la aplicación, y la integración sistemática de la programación no son sugerencias cosméticas, sino los pilares de una reforma necesaria para revitalizar el curso y alinearlos con las exigencias de la formación de un ingeniero informático del siglo XXI.

La Pontificia Universidad Católica del Perú tiene la oportunidad y la responsabilidad de transformar INF134 de un obstáculo temido a una experiencia de aprendizaje fundamental y empoderadora. Perpetuar el modelo actual es un perjuicio para sus estudiantes y un menoscabo a la excelencia académica que la institución proclama. La modernización es el único camino coherente con la misión de formar profesionales capaces de liderar y innovar en el vertiginoso campo de la tecnología.

Referencias

1. [Mathematics for Computer Science \(6.1200J\) - Spring 2024 Syllabus - MIT OpenCourseWare](https://ocw.mit.edu/courses/6-1200j-mathematics-for-computer-science-spring-2024/pages/syllabus/) (https://ocw.mit.edu/courses/6-1200j-mathematics-for-computer-science-spring-2024/pages/syllabus/)
2. [CS103: Mathematical Foundations of Computing - Stanford University](https://web.stanford.edu/class/cs103/) (https://web.stanford.edu/class/cs103/)
3. [21-127 Concepts of Mathematics - Carnegie Mellon University](https://www.math.cmu.edu/~bwsul-liv/teaching/concepts/syllabus.pdf) (https://www.math.cmu.edu/~bwsul-liv/teaching/concepts/syllabus.pdf)
4. [Discrete Mathematics and Probability Theory - University of California, Berkeley](https://www.eecs70.org/) (https://www.eecs70.org/)
5. [Computer Science 20: Discrete Mathematics for Computer Science - Harvard University](https://lewis.seas.harvard.edu/pages/harvard-computer-science-20-discrete-mathematics-computer-science) (https://lewis.seas.harvard.edu/pages/harvard-computer-science-20-discrete-mathematics-computer-science)
6. [INF134 - ESTRUCTURAS DISCRETAS - Pontificia Universidad Católica del Perú \(PUCP\)](https://www.cs.cornell.edu/courses/cs2800/2017sp/syllabus.html) (https://www.cs.cornell.edu/courses/cs2800/2017sp/syllabus.html)
7. [Computing Curricula 2020 \(CC2020\) - Association for Computing Machinery](https://www.acm.org/binaries/content/assets/education/curricula-recommendations/cc2020.pdf) (https://www.acm.org/binaries/content/assets/education/curricula-recommendations/cc2020.pdf)
8. [ACM and IEEE Computer Society Release Computing Curricula 2020 - Association for Computing Machinery](https://www.acm.org/media-center/2021/march/computing-curricula-2020) (https://www.acm.org/media-center/2021/march/computing-curricula-2020)
9. [Computer Science Curricula 2013 - Association for Computing Machinery](https://www.acm.org/binaries/content/assets/education/cs2013_web_final.pdf) (https://www.acm.org/binaries/content/assets/education/cs2013_web_final.pdf)
10. [Rote Memorization is Outdated: Why Teachers Must Rethink Math & CS Education - LinkedIn](https://www.linkedin.com/pulse/rote-memorization-outdated-why-teachers-must-math-cs-philippe-bourdon-i4wte) (https://www.linkedin.com/pulse/rote-memorization-outdated-why-teachers-must-math-cs-philippe-bourdon-i4wte)
11. [From Rules to Reasoning: Why We Need Thinkers, Not Memorizers in Math Education - MIND Education](https://www.mindeducation.org/resources/from-rules-to-reasoning-why-we-need-thinkers-not-memorizers-in-math-education/) (https://www.mindeducation.org/resources/from-rules-to-reasoning-why-we-need-thinkers-not-memorizers-in-math-education/)
12. [Matemática Discreta - MA265 - 202302 - Repositorio Académico UPC](https://repositorioacademico.upc.edu.pe/bitstream/handle/10757/679924/MA265_Matematica_Discreta_202302.pdf) (https://repositorioacademico.upc.edu.pe/bitstream/handle/10757/679924/MA265_Matematica_Discreta_202302.pdf)
13. [Carrera de Ciencia de la Computación - Malla Curricular - UTEC](https://www1.utec.edu.pe/carreras/ciencia-de-la-computacion/malla-curricular) (https://www1.utec.edu.pe/carreras/ciencia-de-la-computacion/malla-curricular)
14. [CS1D01.pdf - cs.utec.edu.pe](https://cs.utec.edu.pe/assets/files/syllabus/2020-II/CS1D01.pdf) (https://cs.utec.edu.pe/assets/files/syllabus/2020-II/CS1D01.pdf)
15. [PLAN_ESTUDIOS-_INGENIERIA-SOFTWARE_11-07-2021-CF-V1.pdf - fiis.uni.edu.pe](https://www.fiis.uni.edu.pe/images/contenidos/escuela-software/PLAN_ESTUDIOS-_INGENIERIA-SOFTWARE_11-07-2021-CF-V1.pdf) (https://www.fiis.uni.edu.pe/images/contenidos/escuela-software/PLAN_ESTUDIOS-_INGENIERIA-SOFTWARE_11-07-2021-CF-V1.pdf)
16. [SUMILLAS.pdf - matematicas.unmsm.edu.pe](https://matematicas.unmsm.edu.pe/img/about/ep-computacion-cientifica/SUMILLAS.pdf) (https://matematicas.unmsm.edu.pe/img/about/ep-computacion-cientifica/SUMILLAS.pdf)
17. [I_2_MAT_DIS.pdf - web.unfv.edu.pe](https://web.unfv.edu.pe/facultades/fiei/images/escuelas/epii/silabos/plan_2019/02_sem/I_2_MAT_DIS.pdf) (https://web.unfv.edu.pe/facultades/fiei/images/escuelas/epii/silabos/plan_2019/02_sem/I_2_MAT_DIS.pdf)
18. [Estructuras discretas en computación - Universidad de Lima](https://www.ulima.edu.pe/publicaciones/estructuras-discretas-en-computacion) (https://www.ulima.edu.pe/publicaciones/estructuras-discretas-en-computacion)
19. [The false dichotomy between lecture and active learning: a quantitative and qualitative investigation of student learning in an undergraduate STEM course - AIMS Press](https://www.aimspress.com/aimspress-data/steme/2022/4/PDF/2767-1925_2022_4_275.pdf) (https://www.aimspress.com/aimspress-data/steme/2022/4/PDF/2767-1925_2022_4_275.pdf)
20. [7 Ways to Develop Conceptual Understanding In The Classroom - Third Space Learning](https://thirdspacelearning.com/us/blog/conceptual-understanding-in-math/) (https://thirdspacelearning.com/us/blog/conceptual-understanding-in-math/)

21. [Rote Memorization: Is It Effective In Education? - Third Space Learning](https://thirdspacelearning.com/us/blog/rote-memorization/) (<https://thirdspacelearning.com/us/blog/rote-memorization/>)
22. [Integrating Computational Thinking in a Discrete Structures Course: A Case Study - SpringerLink](https://link.springer.com/chapter/10.1007/978-3-319-52691-1_7) (https://link.springer.com/chapter/10.1007/978-3-319-52691-1_7)
23. [ITiCSE '19: Proceedings of the 24th Annual Conference on Innovation and Technology in Computer Science Education - ACM Digital Library](https://dl.acm.org/doi/proceedings/10.1145/3328778?tocHeading=heading104) (<https://dl.acm.org/doi/proceedings/10.1145/3328778?tocHeading=heading104>)